

FarmTech REST API Documentation

Version: 1.0

Base URL: `http://localhost:3000/api`

Content type: `application/json`

FarmTech provides APIs for account access, agricultural equipment discovery and management, and equipment rental bookings.

Contents

1. [Getting Started](#)
2. [Authentication](#)
3. [Conventions](#)
4. [Endpoint Summary](#)
5. [Endpoint Reference](#)
6. [Data Models](#)
7. [Error Reference](#)

Getting Started

Start the application from the project root:

```
npm install
npm run dev
```

The API is available at `http://localhost:3000/api`. The first request creates `data/farmtech.json` and seeds six equipment records. Run the automated API tests with:

```
npm test
```

Authentication

`GET /health`, `GET /equipment`, and `GET /equipment/:id` are public. All other endpoints require a bearer token.

1. Call `POST /auth/register` or `POST /auth/login`.
2. Read the `token` string from the response.
3. Add this header to protected requests:

```
Authorization: Bearer <token>
```

Passwords are stored as Node.js `bcrypt` hashes. Tokens are random server-side session values and expire after seven days. Password hashes are never returned in API responses.

Conventions

Request format

Send JSON request bodies with the `Content-Type: application/json` header. Dates use ISO calendar format `YYYY-MM-DD`. Equipment prices are positive numbers in Indian rupees per day.

Response format

Single resources are returned as JSON objects. Collections use:

```
{
  "data": [],
  "count": 0
}
```

Errors use:

```
{
  "error": {
    "message": "Human-readable explanation"
  }
}
```

Status codes

Code	Meaning
200	Request completed successfully
201	Resource created successfully
400	Invalid JSON or validation failure
401	Authentication missing or invalid
404	Resource does not exist or is not owned by the caller
409	Duplicate account or booking conflict

Endpoint Summary

Method	Endpoint	Auth	Purpose
GET	/health	No	Check API availability
POST	/auth/register	No	Create an account and session
POST	/auth/login	No	Authenticate an existing account
GET	/equipment	No	Search and filter equipment
GET	/equipment/:id	No	Fetch one equipment item
POST	/equipment	Yes	Add equipment
PATCH	/equipment/:id	Yes	Update equipment
DELETE	/equipment/:id	Yes	Delete equipment
GET	/rentals	Yes	List the caller's rentals
POST	/rentals	Yes	Create a rental booking
GET	/rentals/:id	Yes	Fetch an owned rental
PATCH	/rentals/:id	Yes	Change rental status
DELETE	/rentals/:id	Yes	Delete an owned rental

Endpoint Reference

Health

GET /health

Returns the current service status. No parameters or authentication are required.

Success 200 :

```
{
  "status": "ok",
  "service": "farmtech-api",
  "timestamp": "2026-09-02T12:00:00.000Z"
}
```

Authentication

POST /auth/register

Creates a customer account and returns a session token.

Request body:

Field	Type	Required	Rules
firstName	string	Yes	Must not be blank
lastName	string	Yes	Must not be blank
email	string	Yes	Must be a valid email; normalized to lowercase
phone	string	Yes	Must not be blank
password	string	Yes	Minimum 8 characters

Example request:

```
{
  "firstName": "Asha",
  "lastName": "Patel",
  "email": "asha@example.com",
  "phone": "+91 9876543210",
  "password": "secure-pass-123"
}
```

Success 201 :

```
{
  "user": {
    "id": "uuid",
    "firstName": "Asha",
    "lastName": "Patel",
    "email": "asha@example.com",
    "phone": "+91 9876543210",
    "role": "customer",
    "createdAt": "2026-09-02T12:00:00.000Z"
  },
  "token": "64-character-session-token"
}
```

Possible errors: 400 invalid or missing fields; 409 email already registered.

POST /auth/login

Authenticates an existing account.

Request body:

Field	Type	Required
email	string	Yes
password	string	Yes

Example request:

```
{
  "email": "asha@example.com",
  "password": "secure-pass-123"
}
```

Success 200 : Returns { "user": { ... }, "token": "..." } using the same user shape as registration.

Possible errors: 400 missing fields; 401 invalid email or password.

Equipment

GET /equipment

Lists equipment and supports optional filtering.

Query parameters:

Parameter	Type	Required	Description
search	string	No	Matches equipment name or type, case-insensitively
location	string	No	Matches the location, case-insensitively
availability	string	No	Exact value: available or unavailable

Example: GET /equipment?search=tractor&location=Punjab&availability=available

Success 200 :

```
{
  "data": [
    {
      "id": "uuid",
      "name": "John Deere Tractor",
      "type": "Tractor",
      "pricePerDay": 1500,
      "rating": 4.8,
      "location": "Punjab",
      "availability": "available",
      "image": "🚜",
      "owner": "FarmTech Demo",
      "createdAt": "2026-09-02T12:00:00.000Z",
      "updatedAt": "2026-09-02T12:00:00.000Z"
    }
  ],
  "count": 1
}
```

GET /equipment/:id

Returns one equipment item.

Path parameter: `id` string, required, equipment UUID.

Success 200 : Returns one equipment object. **Possible error:** `404` equipment not found.

POST /equipment

Creates equipment owned by the authenticated user.

Request body:

Field	Type	Required	Rules
<code>name</code>	string	Yes	Must not be blank
<code>type</code>	string	Yes	Must not be blank
<code>location</code>	string	Yes	Must not be blank
<code>pricePerDay</code>	number	Yes	Must be greater than zero
<code>availability</code>	string	No	<code>available</code> by default; may be <code>unavailable</code>
<code>image</code>	string	No	Defaults to 

Success 201 : Returns the newly created equipment object.

Possible errors: `400` invalid body or price; `401` authentication required.

PATCH /equipment/:id

Updates an equipment item. Authentication is required.

Path parameter: `id` string, required, equipment UUID.

Request body: A partial object containing only these fields: `name` , `type` , `location` , `pricePerDay` , `availability` , `image` . Unknown fields are rejected. When supplied, `pricePerDay` must be a positive number.

Success 200 : Returns the updated equipment object. **Possible errors:** `400` invalid fields; `401` unauthenticated; `404` equipment not found.

DELETE /equipment/:id

Deletes an equipment item.

Path parameter: `id` string, required, equipment UUID.

Success 200 :

```
{ "message": "Equipment deleted" }
```

Possible errors: `401` unauthenticated; `404` equipment not found.

Rentals

GET /rentals

Lists rentals belonging only to the authenticated user. No query or body parameters.

Success 200 :

```
{
  "data": [
    {
      "id": "uuid",
      "userId": "uuid",
      "equipmentId": "uuid",
      "equipmentName": "John Deere Tractor",
      "startDate": "2030-01-01",
      "endDate": "2030-01-03",
      "days": 3,
      "total": 4500,
      "status": "pending",
      "createdAt": "2026-09-02T12:00:00.000Z"
    }
  ]
}
```

Possible error: 401 authentication required.

POST /rentals

Creates a pending booking and calculates the rental total from the equipment daily price.

Request body:

Field	Type	Required	Rules
equipmentId	string	Yes	Must identify existing equipment
startDate	string	Yes	Valid date in YYYY-MM-DD format
endDate	string	Yes	Valid date on or after startDate

Example request:

```
{
  "equipmentId": "equipment-uuid",
  "startDate": "2030-01-01",
  "endDate": "2030-01-03"
}
```

Success 201 : Returns the rental object with `days: 3`, `total` equal to `days * pricePerDay`, and `status: "pending"`.

Possible errors: 400 missing/invalid fields or reversed dates; 401 unauthenticated; 404 equipment not found; 409 equipment unavailable or date range overlaps another pending, confirmed, or active rental.

GET /rentals/:id

Returns a rental owned by the authenticated user.

Path parameter: `id` string, required, rental UUID.

Success 200 : Returns one rental object. **Possible errors:** `401` unauthenticated; `404` rental not found or not owned by caller.

PATCH /rentals/:id

Changes the status of an owned rental.

Path parameter: `id` string, required, rental UUID.

Request body:

Field	Type	Required	Allowed value
<code>status</code>	string	Yes	<code>pending</code> or <code>cancelled</code>

Success 200 : Returns the updated rental object. **Possible errors:** `400` unsupported status; `401` unauthenticated; `404` rental not found or not owned by caller.

DELETE /rentals/:id

Deletes an owned rental.

Path parameter: `id` string, required, rental UUID.

Success 200 :

```
{ "message": "Rental deleted" }
```

Possible errors: `401` unauthenticated; `404` rental not found or not owned by caller.

Data Models

User

Field	Type
<code>id</code>	UUID string
<code>firstName</code> , <code>lastName</code> , <code>email</code> , <code>phone</code>	string
<code>role</code>	<code>customer</code>
<code>createdAt</code>	ISO timestamp

Equipment

Field	Type
id	UUID string
name , type , location , image , owner	string
pricePerDay , rating	number
availability	available or unavailable
createdAt , updatedAt	ISO timestamp

Rental

Field	Type
id , userId , equipmentId	UUID string
equipmentName	string snapshot
startDate , endDate	YYYY-MM-DD string
days , total	number
status	pending , confirmed , active , or cancelled
createdAt	ISO timestamp

Error Reference

All errors use the same envelope:

```
{
  "error": {
    "message": "Equipment is already booked for these dates"
  }
}
```

The API does not expose stack traces, password hashes, or internal database details to clients. The local JSON store can be relocated with the `FARMTECH_DB_FILE` environment variable.